

TEMPORAL GRAPH NETWORKS FOR DEEP LEARNING ON DYNAMIC GRAPHS

Emanuele Rossi*
Twitter

Ben Chamberlain
Twitter

Fabrizio Frasca
Twitter

Davide Eynard
Twitter

Federico Monti
Twitter

Michael Bronstein
Twitter

ABSTRACT

Graph Neural Networks (GNNs) have recently become increasingly popular due to their ability to learn complex systems of relations or interactions. These arise in a broad spectrum of problems ranging from biology and particle physics to social networks and recommendation systems. Despite the plethora of different models for deep learning on graphs, few approaches have been proposed for dealing with graphs that are dynamic in nature (e.g. evolving features or connectivity over time). We present Temporal Graph Networks (TGNs), a generic, efficient framework for deep learning on dynamic graphs represented as sequences of timed events. Thanks to a novel combination of memory modules and **graph-based operators**, TGNs significantly outperform previous approaches while being more computationally efficient. We furthermore show that several previous models for learning on dynamic graphs can be cast as specific instances of our framework. We perform a detailed **ablation study** of different components of our framework and devise the best configuration that achieves state-of-the-art performance on several **transductive and inductive prediction** tasks for dynamic graphs.

1 INTRODUCTION

In the past few years, graph representation learning (Bronstein et al., 2017; Hamilton et al., 2017b; Battaglia et al., 2018) has produced a sequence of successes, gaining increasing popularity in machine learning. Graphs are ubiquitously used as models for systems of relations and interactions in many fields (Battaglia et al., 2016; Qi et al., 2018; Monti et al., 2016; Choma et al., 2018; Duvenaud et al., 2015; Gilmer et al., 2017; Parisot et al., 2018; Rossi et al., 2019), in particular, social sciences (Ying et al., 2018; Monti et al., 2019; Rossi et al., 2020) and biology (Zitnik et al., 2018; Veselkov et al., 2019; Gainza et al., 2019). Learning on such data is possible using graph neural networks (GNNs) (Hamilton et al., 2017a) that typically operate by a **message passing mechanism** (Battaglia et al., 2018) aggregating information in a neighborhood of a node and create node embeddings that are then used for node classification (Monti et al., 2016; Velickovic et al., 2018; Kipf & Welling, 2017), graph classification (Gilmer et al., 2017), or edge prediction (Zhang & Chen, 2018) tasks.

The majority of methods for deep learning on graphs assume that the underlying **graph is static**. However, most real-life systems of interactions such as social networks or biological interactomes are *dynamic*. While it is often possible to apply static graph deep learning models (Liben-Nowell & Kleinberg, 2007) to dynamic graphs by **ignoring the temporal evolution**, this has been shown to be sub-optimal (Xu et al., 2020), and in some cases, it is the dynamic structure that contains crucial insights about the system. Learning on dynamic graphs is relatively recent, and most works are limited to the setting of **discrete-time dynamic graphs represented as a sequence of snapshots** of the graph (Liben-Nowell & Kleinberg, 2007; Dunlavy et al., 2011; Yu et al., 2019; Sankar et al., 2020; Pareja et al., 2019; Yu et al., 2018). Such approaches are unsuitable for interesting real world settings such as social networks, where dynamic graphs are continuous (i.e. edges can appear at any time)

*erossi@twitter.com

and evolving (i.e. new nodes join the graph continuously). Only recently, several approaches have been proposed that support the **continuous-time scenario** (Xu et al., 2020; Trivedi et al., 2019; Kumar et al., 2019; Ma et al., 2018; Nguyen et al., 2018; Bastas et al., 2019).

Contributions. In this paper, we first propose the generic **inductive framework** of Temporal Graph Networks (TGNs) operating on continuous-time dynamic graphs represented as a sequence of events, and show that many previous methods are specific instances of TGNs. Second, we propose a novel training strategy allowing the model to learn from the **sequentiality of the data** while **maintaining highly efficient parallel processing**. Third, we perform a detailed ablation study of different components of our framework and analyze the tradeoff between speed and accuracy. Finally, we show state-of-the-art performance on multiple tasks and datasets in both transductive and inductive settings, while being much faster than previous methods.

2 BACKGROUND

Deep learning on static graphs. A static graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ comprises nodes $\mathcal{V} = \{1, \dots, n\}$ and edges $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$, which are endowed with features, denoted by $\mathbf{v}_i$ and $\mathbf{e}_{ij}$ for all $i, j = 1, \dots, n$, respectively. A typical *graph neural network* (GNN) creates an *embedding* $\mathbf{z}_i$ of the nodes by learning a local aggregation rule of the form

$$\mathbf{z}_i = \sum_{j \in \mathcal{N}_i} h(\mathbf{m}_{ij}, \mathbf{v}_i) \quad \mathbf{m}_{ij} = \text{msg}(\mathbf{v}_i, \mathbf{v}_j, \mathbf{e}_{ij}),$$

which is interpreted as message passing from the neighbors j of i . Here, $\mathcal{N}_i = \{j : (i, j) \in \mathcal{E}\}$ denotes the neighborhood of node i and msg and h are learnable functions.

Dynamic Graphs. There exist two main models for dynamic graphs. *Discrete-time dynamic graphs* (DTDG) are sequences of static graph snapshots taken at intervals in time. *Continuous-time dynamic graphs* (CTDG) are more general and can be represented as timed lists of events, which may include edge addition or deletion, node addition or deletion and node or edge feature transformations.

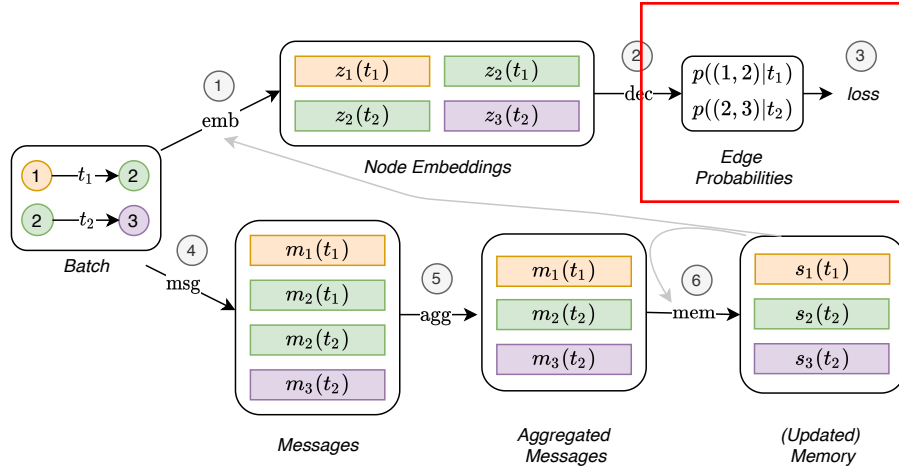
Our temporal (multi-)graph is modeled as a sequence of time-stamped *events* $\mathcal{G} = \{x(t_1), x(t_2), \dots\}$, representing addition or change of a node or interaction between a pair of nodes at times $0 \leq t_1 \leq t_2 \leq \dots$. An event $x(t)$ can be of two types: 1) A **node-wise event** is represented by $\mathbf{v}_i(t)$, where i denotes the index of the node and $\mathbf{v}$ is the vector attribute associated with the event. If the index i has not **been seen before, the event creates node i** (with the given features), **otherwise it updates** the features. 2) An **interaction event** between nodes i and j is represented by a (directed) *temporal edge* $\mathbf{e}_{ij}(t)$ (there might be more than one edge between a pair of nodes, so technically this is a multigraph). We denote by $\mathcal{V}(T) = \{i : \exists \mathbf{v}_i(t) \in \mathcal{G}, t \in T\}$ and $\mathcal{E}(T) = \{(i, j) : \exists \mathbf{e}_{ij}(t) \in \mathcal{G}, t \in T\}$ the temporal set of vertices and edges, respectively, and by $\mathcal{N}_i(T) = \{j : (i, j) \in \mathcal{E}(T)\}$ the neighborhood of node i in time interval T . $\mathcal{N}_i^k(T)$ denotes the k -hop neighborhood. A *snapshot* of the temporal graph $\mathcal{G}$ at time t is the (multi-)graph $\mathcal{G}(t) = (\mathcal{V}[0, t], \mathcal{E}[0, t])$ with $n(t)$ nodes. **Deletion events** are discussed in Appendix A.1.

3 TEMPORAL GRAPH NETWORKS

Following the terminology in (Kazemi et al., 2020), a neural model for dynamic graphs can be regarded as an encoder-decoder pair, where an encoder is a function that maps from a dynamic graph to node embeddings, and a decoder takes as input one or more node embeddings and makes a task-specific prediction e.g. node classification or edge prediction. The key contribution of this paper is a novel Temporal Graph Network (TGN) **encoder** applied on a continuous-time dynamic graph represented as a sequence of time-stamped events and producing, for **each time t , the embedding** of the graph nodes $\mathbf{Z}(t) = (\mathbf{z}_1(t), \dots, \mathbf{z}_{n(t)}(t))$.

3.1 CORE MODULES

Memory. The memory (state) of the model at time t consists of a vector $\mathbf{s}_i(t)$ for each node i the model has seen so far. The memory of a node is updated after an event (e.g. interaction with another node or node-wise change), and its purpose is to represent the node’s **history in a compressed format**.



不太懂这里算出边的概率怎么算，跟文本说的好像是不一样的

Figure 1: Computations performed by TGN on a batch of time-stamped interactions. *Top*: embeddings are produced by the embedding module using the temporal graph and the node’s memory (1). The embeddings are then used to predict the batch interactions and compute the loss (2, 3). *Bottom*: these same interactions are used to update the memory (4, 5, 6). This is a simplified flow of operations which would prevent the training of all the modules in the bottom as they would **not receiving a gradient**. Section 3.2 explains how to change the flow of operations to solve this problem and figure 2 shows the complete diagram.

Thanks to this specific module, TGNs have the capability to memorize long term dependencies for each node in the graph. When a new node is encountered, its memory is initialized as the zero vector, and it is then updated for each event involving the node, even after the model has finished training. While a global (graph-wise) memory can also be added to the model to track the evolution of the entire network, we leave this as future work.

Message Function. For each event involving node i , a message is computed to update i ’s memory. In the case of an interaction event $e_{ij}(t)$ between source node i and target node j at time t , two messages can be computed:

$$\mathbf{m}_i(t) = \text{msg}_s(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_d(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t)) \quad (1)$$

Similarly, in case of a node-wise event $\mathbf{v}_i(t)$, a single message can be computed for the node involved in the event:

$$\mathbf{m}_i(t) = \text{msg}_n(\mathbf{s}_i(t^-), t, \mathbf{v}_i(t)). \quad (2)$$

Here, $\mathbf{s}_i(t^-)$ is the memory of node i just before time t (i.e., from the time of the previous interaction involving i) and msg_s , msg_d and msg_n are learnable message functions, e.g. MLPs. In all experiments, we choose the message function as *identity* (id), which is simply the concatenation of the inputs, for the sake of simplicity. Deletion events are also supported by the framework and are presented in Appendix A.1. A more complex message function that involves additional aggregation from the neighbours of nodes i and j is also possible and is left for future study.

Message Aggregator. Resorting to batch processing for efficiency reasons may lead to multiple events involving the same node i in the same batch. As each event generates a message in our formulation, we use a mechanism to aggregate messages $\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)$ for $t_1, \dots, t_b \leq t$,

$$\bar{\mathbf{m}}_i(t) = \text{agg}(\mathbf{m}_i(t_1), \dots, \mathbf{m}_i(t_b)). \quad (3)$$

Here, agg is an aggregation function. While multiple choices can be considered for implementing this module (e.g. RNNs or attention w.r.t. the node memory), for the sake of simplicity we considered two efficient non-learnable solutions in our experiments: *most recent message* (keep only most recent message for a given node) and *mean message* (average all messages for a given node). We leave learnable aggregation as a future research direction.

Memory Updater. As previously mentioned, the memory of a node is updated upon each event involving the node itself:

$$\mathbf{s}_i(t) = \text{mem}(\bar{\mathbf{m}}_i(t), \mathbf{s}_i(t^-)). \quad (4)$$

For interaction events involving **two nodes** i and j , the memories of both nodes are updated after the event has happened. For **node-wise events**, only the memory of the related node is updated. Here, mem is a learnable memory update function, e.g. a recurrent neural network such as LSTM (Hochreiter & Schmidhuber, 1997) or GRU (Cho et al., 2014).

Embedding. The embedding module is used to generate the temporal embedding $\mathbf{z}_i(t)$ of node i at any time t . The main goal of the embedding module is to avoid the so-called memory staleness problem (Kazemi et al., 2020). Since the memory of a node i is updated only when the node is involved in an event, it might happen that, in the absence of events for a long time (e.g. a social network user who stops using the platform for some time before becoming active again), i 's memory becomes stale. While multiple implementations of the embedding module are possible, we use the form:

$$\mathbf{z}_i(t) = \text{emb}(i, t) = \sum_{j \in \mathcal{N}_i^k([0, t])} h(\mathbf{s}_i(t), \mathbf{s}_j(t), \mathbf{e}_{ij}, \mathbf{v}_i(t), \mathbf{v}_j(t)),$$

where h is a learnable function. This includes many different formulations as particular cases:

Identity (id): $\text{emb}(i, t) = \mathbf{s}_i(t)$, which uses the memory directly as the node embedding.

Time projection (time): $\text{emb}(i, t) = (1 + \Delta t \mathbf{w}) \circ \mathbf{s}_i(t)$, where $\mathbf{w}$ are learnable parameters, Δt is the time since the last interaction, and $\circ$ denotes **element-wise vector** product. This version of the embedding method was used in Jodie (Kumar et al., 2019).

Temporal Graph Attention (attn): A series of L graph attention layers compute i 's embedding by aggregating information from its L -hop temporal neighborhood.

The input to the l -th layer is i 's representation $\mathbf{h}_i^{(l-1)}(t)$, the current timestamp t , i 's neighborhood representation $\{\mathbf{h}_1^{(l-1)}(t), \dots, \mathbf{h}_N^{(l-1)}(t)\}$ together with timestamps $t_1, \dots, t_N$ and features $\mathbf{e}_{i1}(t_1), \dots, \mathbf{e}_{iN}(t_N)$ for each of the considered interactions which form an edge in i 's temporal neighborhood:

$$\mathbf{h}_i^{(l)}(t) = \text{MLP}^{(l)}(\mathbf{h}_i^{(l-1)}(t) \parallel \tilde{\mathbf{h}}_i^{(l)}(t)), \quad \text{拼接, 经过mlp, 计算得分}$$

$$\tilde{\mathbf{h}}_i^{(l)}(t) = \text{MultiHeadAttention}^{(l)}(\mathbf{q}^{(l)}(t), \mathbf{K}^{(l)}(t), \mathbf{V}^{(l)}(t)), \quad (6)$$

$$\mathbf{q}^{(l)}(t) = \mathbf{h}_i^{(l-1)}(t) \parallel \phi(0),$$

上一层的表示拼接时间编码, 我记得也可以不用拼接的形式

$$\mathbf{K}^{(l)}(t) = \mathbf{V}^{(l)}(t) = \mathbf{C}^{(l)}(t), \quad (8)$$

$$\mathbf{C}^{(l)}(t) = [\mathbf{h}_1^{(l-1)}(t) \parallel \mathbf{e}_{i1}(t_1) \parallel \phi(t - t_1), \dots, \mathbf{h}_N^{(l-1)}(t) \parallel \mathbf{e}_{iN}(t_N) \parallel \phi(t - t_N)]. \quad (9)$$

Here, $\phi(\cdot)$ represents a generic time encoding (Xu et al., 2020), $\parallel$ is the concatenation operator and $\mathbf{z}_i(t) = \text{emb}(i, t) = \mathbf{h}_i^{(L)}(t)$. Each layer amounts to performing multi-head-attention (Vaswani et al., 2017) where the query ($\mathbf{q}^{(l)}(t)$) is a reference node (i.e. the target node or one of its $L - 1$ -hop neighbors), and the keys $\mathbf{K}^{(l)}(t)$ and values $\mathbf{V}^{(l)}(t)$ are its neighbors. Finally, an MLP is used to combine the reference node representation with the aggregated information. Differently from the original formulation of this layer (firstly proposed in TGAT (Xu et al., 2020)) where no node-wise temporal features were used, in our case the input representation of each node $\mathbf{h}_j^{(0)}(t) = \mathbf{s}_j(t) + \mathbf{v}_j(t)$ and as such it allows the model to exploit both the current memory $\mathbf{s}_j(t)$ and the temporal node features $\mathbf{v}_j(t)$.

Temporal Graph Sum (sum): A simpler and faster aggregation over the graph:

$$\mathbf{h}_i^{(l)}(t) = \mathbf{W}_2^{(l)}(\mathbf{h}_i^{(l-1)}(t) \parallel \tilde{\mathbf{h}}_i^{(l)}(t)), \quad (10)$$

$$\tilde{\mathbf{h}}_i^{(l)}(t) = \text{ReLU}\left(\sum_{j \in \mathcal{N}_i([0, t])} \mathbf{W}_1^{(l)}(\mathbf{h}_j^{(l-1)}(t) \parallel \mathbf{e}_{ij} \parallel \phi(t - t_j))\right). \quad (11)$$

Here as well, $\phi(\cdot)$ is a time encoding and $\mathbf{z}_i(t) = \text{emb}(i, t) = \mathbf{h}_i^{(L)}(t)$. In the experiment, both for the *Temporal Graph Attention* and for the *Temporal Graph Sum* modules we use the time encoding presented in Time2Vec (Kazemi et al., 2019) and used in TGAT (Xu et al., 2020).

The graph embedding modules mitigate the staleness problem by aggregating information from a node's neighbors memory. When a node has been inactive for a while, it is likely that some of its

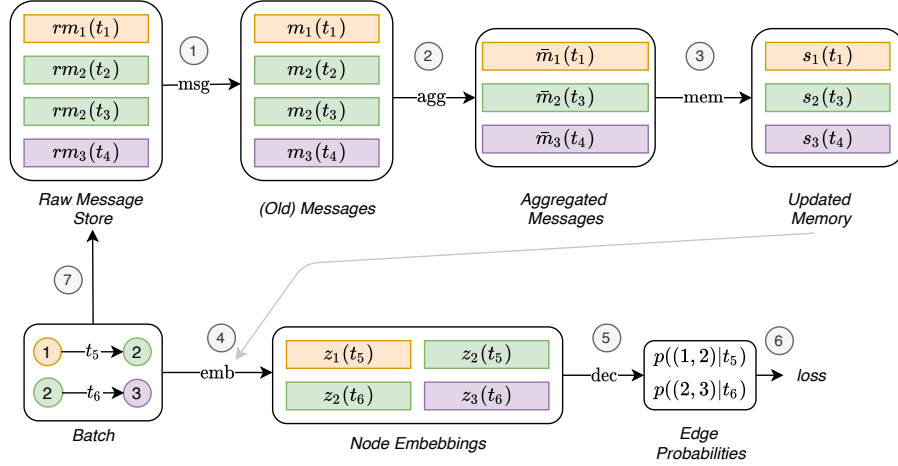


Figure 2: Flow of operations of TGN used to train the memory-related modules. *Raw Message Store* stores the necessary raw information to compute messages, i.e. the input to the message functions, which we call raw messages, for interactions which have been processed by the model in the past. This allows the model to delay the memory update brought by an interaction to later batches. At first, the memory is updated using messages computed from raw messages stored in previous batches (1, 2, 3). The embeddings can then be computed using the just updated memory (grey link) (4). By doing this, the computation of the memory-related modules directly influences the loss (5, 6), and they receive a gradient. Finally, the raw messages for this batch interactions are stored in the raw message store (6) to be used in future batches.

neighbours have been recently active, and by aggregating their memories, TGN can compute an up-to-date embedding for the node. The temporal graph attention is additionally able to select which neighbors are more important based on both features and timing information.

3.2 TRAINING

TGN can be trained for a variety of tasks such as edge prediction (self-supervised) or node classification (semi-supervised). We use link prediction as an example: provided a list of time ordered interactions, the goal is to predict future interactions from those observed in the past. Figure 1 shows the computations performed by TGN on a batch of training data.

The complexity in our training strategy relates to the memory-related modules (*Message function*, *Message aggregator*, and *Memory updater*) because they do not directly influence the loss and therefore do not receive a gradient. To solve this problem, the memory must be updated before predicting the batch interactions. However, updating the memory with an interaction $e_{ij}(t)$ before using the model to predict that same interaction, causes information leakage. To avoid the issue, when processing a batch, we update the memory with messages coming from previous batches (which are stored in the *Raw Message Store*), and then predict the interactions. Figure 2 shows the training flow for the memory-related modules. Pseudocode for the training procedure is presented in Appendix A.2.

More formally, at any time t , the *Raw Message Store* contains (at most) one raw message rm_i for each node i ¹, generated from the last interaction involving i before time t . When the model processes the next interactions involving i , its memory is updated using rm_i (arrows 1, 2, 3 in Figure 2), then the updated memory is used to compute the node’s embedding and the batch loss (arrows 4, 5, 6). Finally, the raw messages for the new interaction are stored in the raw message store (arrows 7). It is also worth noticing that all predictions in a given batch have access to the same state of the memory. While from the perspective of the first interaction in the batch the memory is up-to-date (since it contains information about all previous interactions in the graph), from the perspective of the last interaction in the batch the same memory is out-of-date, since it lacks information about previous interactions in the same batch. This disincentives the use of a big batch size (in the extreme case where the batch size is as big as the dataset, all predictions would be made using the initial zero memory). We found a batch size of 200 to be a good trade-off between speed and update granularity.

¹The *Raw Message Store* does not contain a message for i only if i has never been involved in an event in the past.

4 RELATED WORK

Early models for learning on dynamic graphs focused on DTDGs. Such approaches either aggregate graph snapshots and then apply static methods (Liben-Nowell & Kleinberg, 2007; Hisano, 2018; Sharan & Neville, 2008; Ibrahim & Chen, 2015; Ahmed & Chen, 2016; Ahmed et al., 2016), assemble snapshots into tensors and factorize (Dunlavy et al., 2011; Yu et al., 2017; Ma et al., 2019), or encode each snapshot to produce a series of embeddings. In the latter case, the embeddings are either aggregated by taking a weighted sum (Yao et al., 2016; Zhu et al., 2012), fit to time series models (Huang & Lin, 2009; Güneş et al., 2016; da Silva Soares & Prudêncio, 2012; Moradabadi & Meybodi, 2017), used as components in RNNs (Seo et al., 2018; Narayan & Roe, 2018; Manessi et al., 2020; Yu et al., 2019; Chen et al., 2018; Sankar et al., 2020; Pareja et al., 2019), or learned by imposing a smoothness constraint over time (Kim & Han, 2009; Gupta et al., 2011; Yao et al., 2016; Zhu et al., 2017; Zhou et al., 2018; Singer et al., 2019; Goyal et al., 2018; Fard et al., 2019; Pei et al., 2016). Another line of work encodes DTDGs by first performing random walks on an initial snapshot and then modifying the walk behaviour for subsequent snapshots (Mahdavi et al., 2018; Du et al., 2018; Xin et al., 2016; De Winter et al., 2018; Yu et al., 2018). Spatio-temporal graphs (considered by Zhang et al. (2018); Li et al. (2018) for traffic forecasting) are specific cases of dynamic graphs where the topology of the graph is fixed.

CTDGs have been addressed only recently. Several approaches use random walk models (Nguyen et al., 2018; Nguyen et al., 2018; Bastas et al., 2019) incorporating continuous time through constraints on transition probabilities. Sequence-based approaches for CTDGs (Kumar et al., 2019; Trivedi et al., 2017; 2019; Ma et al., 2018) use RNNs to update representations of the source and destination node each time a new edge appears. Other recent works have focused on dynamic knowledge graphs (Goel et al., 2019; Xu et al., 2019; Dasgupta et al., 2018; García-Durán et al., 2018). Many architectures for continuous-time dynamic graphs are based on a node-wise memory updated by an RNN when new interactions appear. Yet, they lack a GNN-like aggregation from a node’s neighbors when computing its embedding, which makes them susceptible to the staleness problem (i.e. a node embedding becoming out of date) while at the same time also limiting their expressive power.

Most recent CTDGs learning models can be interpreted as specific cases of our framework (see Table 1). For example, Jodie (Kumar et al., 2019) uses the time projection embedding module $\text{emb}(i, t) = (1 + \Delta t \mathbf{w}) \circ \mathbf{s}_i(t)$. TGAT (Xu et al., 2020) is a specific case of TGN when the memory and its related modules are missing, and graph attention is used as the Embedding module. DyRep (Trivedi et al., 2019) computes messages using graph attention on the destination node neighbors. Finally, we note that TGN generalizes the Graph Networks (GN) model (Battaglia et al., 2018) for static graphs (with the exception of the global block that we omitted from our model for the sake of simplicity), and thus the majority of existing graph message passing-type architectures.

Table 1: Previous models for deep learning on continuous-time dynamic graphs are specific case of our TGN framework. Shown are multiple variants of TGN used in our ablation studies. *method* (l, n) refers to graph convolution using l layers and n neighbors. [†]uses t-batches. * uses uniform sampling of neighbors, while the default is sampling the most recent neighbors. [‡]message aggregation not explained in the paper. ^{||} uses a summary of the destination node neighborhood (obtained through graph attention) as additional input to the message function.

	Mem.	Mem. Updater	Embedding	Mess. Agg.	Mess. Func.
Jodie	node	RNN	time	— [†]	id
TGAT	—	—	attn (2l, 20n)*	—	—
DyRep	node	RNN	id	— [‡]	attn
TGN-attn	node	GRU	attn (1l, 10n)	last	id
TGN-2l	node	GRU	attn (2l, 10n)	last	id
TGN-no-mem	—	—	attn (1l, 10n)	—	—
TGN-time	node	GRU	time	last	id
TGN-id	node	GRU	id	last	id
TGN-sum	node	GRU	sum (1l, 10n)	last	id
TGN-mean	node	GRU	attn (1l, 10n)	mean	id

Table 2: Average Precision (%) for future edge prediction task in transductive and inductive settings. **First, Second, Third** best performing method. *Static graph method. †Does not support inductive.

	Wikipedia		Reddit		Twitter	
	Transductive	Inductive	Transductive	Inductive	Transductive	Inductive
GAE*	91.44 $\pm$ 0.1	†	93.23 $\pm$ 0.3	†	—	†
VAGE*	91.34 $\pm$ 0.3	†	92.92 $\pm$ 0.2	†	—	†
DeepWalk*	90.71 $\pm$ 0.6	†	83.10 $\pm$ 0.5	†	—	†
Node2Vec*	91.48 $\pm$ 0.3	†	84.58 $\pm$ 0.5	†	—	†
GAT*	94.73 $\pm$ 0.2	91.27 $\pm$ 0.4	97.33 $\pm$ 0.2	95.37 $\pm$ 0.3	67.57 $\pm$ 0.4	62.32 $\pm$ 0.5
GraphSAGE*	93.56 $\pm$ 0.3	91.09 $\pm$ 0.3	97.65 $\pm$ 0.2	96.27 $\pm$ 0.2	65.79 $\pm$ 0.6	60.13 $\pm$ 0.6
CTDNE	92.17 $\pm$ 0.5	†	91.41 $\pm$ 0.3	†	—	†
Jodie	94.62 $\pm$ 0.5	93.11 $\pm$ 0.4	97.11 $\pm$ 0.3	94.36 $\pm$ 1.1	85.20 $\pm$ 2.4	79.83 $\pm$ 2.5
TGAT	95.34 $\pm$ 0.1	93.99 $\pm$ 0.3	98.12 $\pm$ 0.2	96.62 $\pm$ 0.3	70.02 $\pm$ 0.6	66.35 $\pm$ 0.8
DyRep	94.59 $\pm$ 0.2	92.05 $\pm$ 0.3	97.98 $\pm$ 0.1	95.68 $\pm$ 0.2	83.52 $\pm$ 3.0	78.38 $\pm$ 4.0
TGN-attn	98.46 $\pm$ 0.1	97.81 $\pm$ 0.1	98.70 $\pm$ 0.1	97.55 $\pm$ 0.1	94.52 $\pm$ 0.5	91.37 $\pm$ 1.1

5 EXPERIMENTS

Datasets. We use three datasets in our experiments: Wikipedia, Reddit (Kumar et al., 2019), and Twitter, which are described in detail in Appendix A.3. Our experimental setup closely follows (Xu et al., 2020) and focuses on the tasks of future edge (‘link’) prediction and dynamic node classification. In future edge prediction, the goal is to predict the probability of an edge occurring between two nodes at a given time. Our encoder is combined with a simple MLP decoder mapping from the concatenation of two node embeddings to the probability of the edge. We study both the transductive and inductive settings. In the transductive task, we predict future links of the nodes observed during training, whereas in the inductive tasks we predict future links of nodes never observed before. For node classification, the transductive setting is used. For all tasks and datasets we perform the same 70%-15%-15% chronological split as in Xu et al. (2020). All the results were averaged over 10 runs. Hyperparameters and additional details can be found in Appendix A.4.

Baselines. Our strong baselines are state-of-the-art approaches for continuous time dynamic graphs (CTDNE (Nguyen et al., 2018), Jodie (Kumar et al., 2019), DyRep (Trivedi et al., 2019) and TGAT (Xu et al., 2020)) as well as state-of-the-art models for static graphs (GAE (Kipf & Welling, 2016), VGAE (Kipf & Welling, 2016), DeepWalk (Perozzi et al., 2014), Node2Vec (Grover & Leskovec, 2016), GAT (Velickovic et al., 2018) and GraphSAGE (Hamilton et al., 2017b)).

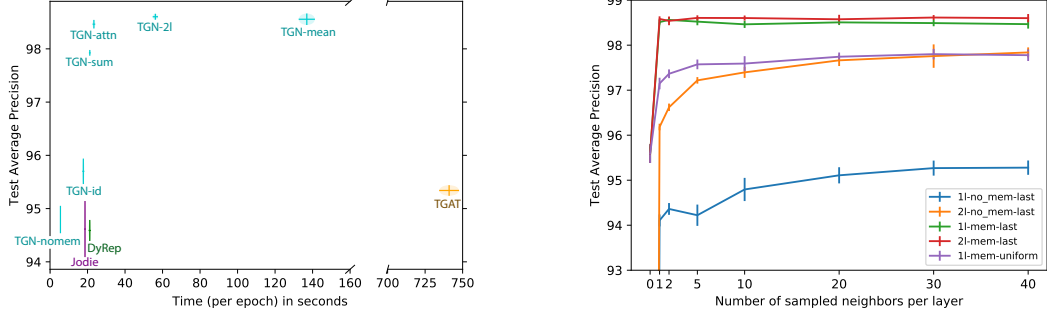
5.1 PERFORMANCE

Table 2 presents the results on future edge prediction. Our model clearly outperforms the baselines by a large margin in both transductive and inductive settings on all datasets. The gap is particularly large on the Twitter dataset, where we outperform the second-best method (DyRep) by over 4% and 10% in the transductive and inductive case respectively. Table 3 shows the results on dynamic node classification, where again our model obtains state-of-the-art results, with a large improvement over all other methods.

Due to the efficient parallel processing and the need for only one graph attention layer (see Section 5.2 for the ablation study on the number of layers), our model is up to 30 $\times$ faster than TGAT per epoch (Figure 3a), while requiring a similar number of epochs to converge.

Table 3: ROC AUC % for the dynamic node classification. *Static graph method.

	Wikipedia	Reddit
GAE*	74.85 $\pm$ 0.6	58.39 $\pm$ 0.5
VAGE*	73.67 $\pm$ 0.8	57.98 $\pm$ 0.6
GAT*	82.34 $\pm$ 0.8	64.52 $\pm$ 0.5
GraphSAGE*	82.42 $\pm$ 0.7	61.24 $\pm$ 0.6
CTDNE	75.89 $\pm$ 0.5	59.43 $\pm$ 0.6
JODIE	84.84 $\pm$ 1.2	61.83 $\pm$ 2.7
TGAT	83.69 $\pm$ 0.7	65.56 $\pm$ 0.7
DyRep	84.59 $\pm$ 2.2	62.91 $\pm$ 2.4
TGN-attn	87.81 $\pm$ 0.3	67.06 $\pm$ 0.9



(a) Tradeoff between accuracy (test average precision in %) and speed (time per epoch in sec) of different models.

(b) Performance of methods with different number of layers and with or without memory when sampling an increasing number of neighbors. *Last* and *uniform* refer to neighbor sampling strategy.

Figure 3: Ablation studies on the Wikipedia dataset for the transductive setting of the future edge prediction task. Means and standard deviations were computed over 10 runs.

5.2 CHOICE OF MODULES

We perform a detailed ablation study comparing different instances of our TGN framework, focusing on the speed vs accuracy tradeoff resulting from the choice of modules and their combination. The variants we experiment with are reported in Table 1 and their results are depicted in Figure 3a.

Memory. We compare a model that does not make use of a memory (TGN-no-mem), with a model which uses memory (TGN-attn) but is otherwise identical. While TGN-attn is about $3\times$ slower, it has nearly 4% higher precision than TGN-no-mem, confirming the importance of memory for learning on dynamic graphs, due to its ability to store long-term information about a node which is otherwise hard to capture. This finding is confirmed in Figure 3b where we compare different models when increasing the number of sampled neighbors: the models with memory consistently outperform the models without memory. Moreover, using the memory in conjunction with sampling the most recent neighbors reduces the number of neighbors needed to achieve the best performance when used.

Embedding Module. Figure 3a compare models with different embedding modules (TGN-id, TGN-time, TGN-attn, TGN-sum). The first interesting insight is that projecting the embedding in time seems to slightly hurt, as TGN-time underperforms TGN-id. Moreover, the ability to exploit the graph is crucial for performance: we note that all graph-based projections (TGN-attn, TGN-sum) outperform the graph-less TGN-id model by a large margin, with TGN-attn being the top performer at the expense of being only slightly slower than the simpler TGN-sum. This indicates that the ability to obtain more recent information through the graph, and to select which neighbors are the most important are critical factors for the performance of the model.

Message Aggregator. We compare two further models, one using the last message aggregator (TGN-attn) and another a mean aggregator (TGN-mean) but otherwise the same. While TGN-mean performs slightly better, it is more than $3\times$ slower.

Number of layers. While in TGAT having 2 layers is of fundamental importance for obtaining good performances (TGAT vs TGAT-1l has over 10% difference in average precision), in TGN the presence of the memory makes it enough to use just one layer to obtain very high performance (TGN-attn vs TGN-2l). This is because when accessing the memory of the 1-hop neighbors, we are indirectly accessing information from hops further away. Moreover, being able to use only one layer of graph attention speeds up the model dramatically.

6 CONCLUSION

We introduce TGN, a generic framework for learning on continuous-time dynamic graphs. We obtain state-of-the-art results on several tasks and datasets while being faster than previous methods.

Detailed ablation studies show the importance of the memory and its related modules to store long-term information, as well as the importance of the graph-based embedding module to generate up-to-date node embeddings. We envision interesting applications of TGN in the fields of social sciences, recommender systems, and biological interaction networks, opening up a future research direction of exploring more advanced settings of our model and understanding the most appropriate domain-specific choices.

REFERENCES

- Nahla Mohamed Ahmed and Ling Chen. An efficient algorithm for link prediction in temporal uncertain social networks. *Information Sciences*, 331:120–136, 2016.
- Nahla Mohamed Ahmed, Ling Chen, Yulong Wang, Bin Li, Yun Li, and Wei Liu. Sampling-based algorithm for link prediction in temporal networks. *Information Sciences*, 374:1–14, 2016.
- Nikolaos Bastas, Theodoros Semertzidis, Apostolos Axenopoulos, and Petros Daras. evolve2vec: Learning network representations using temporal unfolding. In *International Conference on Multimedia Modeling*, pp. 447–458. Springer, 2019.
- Peter W Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, et al. Interaction networks for learning about objects, relations and physics. In *NIPS*, pp. 4502–4510, 2016.
- Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, and Ryan Faulkner. Relational inductive biases, deep learning, and graph networks. *arXiv:1806.01261*, 2018.
- Luca Belli, Sofia Ira Ktena, Alykhan Tejani, Alexandre Lung-Yut-Fon, Frank Portman, Xiao Zhu, Yuanpu Xie, Akshay Gupta, Michael M. Bronstein, Amra Delić, et al. Privacy-preserving recommender systems challenge on twitter’s home timeline. *arXiv:2004.13715*, 2020.
- Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. Geometric deep learning: going beyond euclidean data. *IEEE Signal Process. Mag.*, 34(4):18–42, 2017.
- Jinyin Chen, Xuanheng Xu, Yangyang Wu, and Haibin Zheng. Gc-lstm: Graph convolution embedded lstm for dynamic link prediction. *arXiv:1812.04206*, 2018.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *EMNLP*, pp. 1724–1734, 2014. doi: 10.3115/v1/D14-1179. URL <https://www.aclweb.org/anthology/D14-1179>.
- Nicholas Choma, Federico Monti, Lisa Gerhardt, Tomasz Palczewski, Zahra Ronaghi, Prabhat Prabhat, Wahid Bhimji, Michael M. Bronstein, Spencer Klein, and Joan Bruna. Graph neural networks for iccube signal classification. In *ICMLA*, 2018.
- Paulo Ricardo da Silva Soares and Ricardo Bastos Cavalcante Prudêncio. Time series based link prediction. In *IJCNN*, pp. 1–7. IEEE, 2012.
- Shib Sankar Dasgupta, Swayambhu Nath Ray, and Partha Talukdar. HyTE: Hyperplane-based temporally aware knowledge graph embedding. In *EMNLP*, pp. 2001–2011, 2018. doi: 10.18653/v1/D18-1225. URL <https://www.aclweb.org/anthology/D18-1225>.
- S. De Winter, T. Decuyper, S. Mitrović, B. Baesens, and J. De Weerd. Combining temporal aspects of dynamic networks with node2vec for a more efficient dynamic link prediction. In *ASONAM*, pp. 1234–1241, 2018.
- Lun Du, Yun Wang, Guojie Song, Zhicong Lu, and Junshan Wang. Dynamic network embedding: An extended approach for skip-gram based network embedding. In *IJCAI*, pp. 2086–2092, 2018.
- Daniel M Dunlavy, Tamara G Kolda, and Evrim Acar. Temporal link prediction using matrix and tensor factorizations. *TKDD*, 5(2):1–27, 2011.

-
- David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarelli, Timothy Hirzel, Alan Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. In *NIPS*. 2015.
- Amin Milani Fard, Ebrahim Bagheri, and Ke Wang. Relationship prediction in dynamic heterogeneous information networks. In *European Conference on Information Retrieval*, pp. 19–34. Springer, 2019.
- Pablo Gainza et al. Deciphering interaction fingerprints from protein molecular surfaces using geometric deep learning. *Nature Methods*, 17:184–192, 2019.
- Alberto García-Durán, Sebastijan Dumančić, and Mathias Niepert. Learning sequence encoders for temporal knowledge graph completion. In *EMNLP*, pp. 4816–4821, 2018. doi: 10.18653/v1/D18-1516.
- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *ICML*, 2017.
- Rishab Goel, Seyed Mehran Kazemi, Marcus Brubaker, and Pascal Poupart. Diachronic embedding for temporal knowledge graph completion. *arXiv:1907.03143*, 2019.
- Palash Goyal, Nitin Kamra, Xinran He, and Yan Liu. Dyngem: Deep embedding method for dynamic graphs. *arXiv:1805.11273*, abs/1805.11273, 2018. URL <http://dblp.uni-trier.de/db/journals/corr/corr1805.html#abs-1805-11273>.
- Aditya Grover and Jure Leskovec. Node2vec: Scalable feature learning for networks. In *KDD '16*, KDD '16, pp. 855–864, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450342322. doi: 10.1145/2939672.2939754. URL <https://doi.org/10.1145/2939672.2939754>.
- İsmail Güneş, Şule Gündüz-Öğüdücü, and Zehra Çataltepe. Link prediction using time series of neighborhood-based node similarity scores. *Data Mining and Knowledge Discovery*, 30(1): 147–180, 2016.
- Manish Gupta, Charu C Aggarwal, Jiawei Han, and Yizhou Sun. Evolutionary clustering and analysis of bibliographic networks. In *ASONAM*, pp. 63–70. IEEE, 2011.
- William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *NIPS*, 2017a.
- William L Hamilton, Rex Ying, and Jure Leskovec. Representation learning on graphs: Methods and applications. *arXiv:1709.05584*, 2017b.
- Ryohei Hisano. Semi-supervised graph embedding approach to dynamic link prediction. *Springer Proceedings in Complexity*, pp. 109–121, 2018. ISSN 2213-8692. doi: 10.1007/978-3-319-73198-8_10. URL http://dx.doi.org/10.1007/978-3-319-73198-8_10.
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8): 1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Zan Huang and Dennis KJ Lin. The time-series link prediction problem with applications in communication surveillance. *INFORMS Journal on Computing*, 21(2):286–303, 2009.
- Nahla Mohamed Ahmed Ibrahim and Ling Chen. Link prediction in dynamic social networks by integrating different types of information. *Applied Intelligence*, 42(4):738–750, 2015.
- Seyed Mehran Kazemi, Rishab Goel, Sepehr Eghbali, Janahan Ramanan, Jaspreet Sahota, Sanjay Thakur, Stella Wu, Cathal Smyth, Pascal Poupart, and Marcus Brubaker. Time2vec: Learning a vector representation of time. *CoRR*, abs/1907.05321, 2019. URL <http://arxiv.org/abs/1907.05321>.

-
- Seyed Mehran Kazemi, Rishab Goel, Kshitij Jain, Ivan Kobyzev, Akshay Sethi, Peter Forsyth, and Pascal Poupard. Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70):1–73, 2020. URL <http://jmlr.org/papers/v21/19-447.html>.
- Min-Soo Kim and Jiawei Han. A particle-and-density based evolutionary clustering method for dynamic networks. *VLDB*, 2(1):622–633, 2009.
- Thomas N Kipf and Max Welling. Variational graph auto-encoders. *NIPS Workshop on Bayesian Deep Learning*, 2016.
- Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*, 2017. URL <https://openreview.net/forum?id=SJU4ayYgl>.
- Srijan Kumar, Xikun Zhang, and Jure Leskovec. Predicting dynamic embedding trajectory in temporal interaction networks. In *KDD '19*, pp. 1269–1278, 2019. ISBN 9781450362016. doi: 10.1145/3292500.3330895. URL <https://doi.org/10.1145/3292500.3330895>.
- Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations*, 2018. URL <https://openreview.net/forum?id=SJiHXGWAZ>.
- David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *J. Am. Soc. Inf. Sci. Technol.*, 58(7):1019–1031, May 2007. ISSN 1532-2882.
- Yao Ma, Ziyi Guo, Zhaochun Ren, Eric Zhao, Jiliang Tang, and Dawei Yin. Streaming graph neural networks. *arXiv:1810.10627*, 2018.
- Yunpu Ma, Volker Tresp, and Erik A Daxberger. Embedding models for episodic knowledge graphs. *Journal of Web Semantics*, 59:100490, 2019.
- Sedigheh Mahdavi, Shima Khoshraftar, and Aijun An. dynnode2vec: Scalable dynamic network embedding. In *2018 IEEE International Conference on Big Data*, pp. 3762–3765. IEEE, 2018.
- Franco Manessi, Alessandro Rozza, and Mario Manzo. Dynamic graph convolutional networks. *Pattern Recognition*, 97:107000, 2020.
- Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M. Bronstein. Geometric deep learning on graphs and manifolds using mixture model cnns. In *CVPR*, 2016.
- Federico Monti, Fabrizio Frasca, Davide Eynard, Damon Mannion, and Michael M. Bronstein. Fake news detection on social media using geometric deep learning. *arXiv:1902.06673*, 2019. URL <http://arxiv.org/abs/1902.06673>.
- Behnaz Moradabadi and Mohammad Reza Meybodi. A novel time series link prediction method: Learning automata approach. *Physica A: Statistical Mechanics and its Applications*, 482:422–432, 2017.
- Apurva Narayan and Peter HO’N Roe. Learning graph dynamics using deep neural networks. *IFAC-PapersOnLine*, 51(2):433–438, 2018.
- G. H. Nguyen, J. Boaz Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim. Dynamic network embeddings: From random walks to temporal random walks. In *2018 IEEE International Conference on Big Data*, pp. 1085–1092, 2018.
- Giang Hoang Nguyen, John Boaz Lee, Ryan A. Rossi, Nesreen K. Ahmed, Eunye Koh, and Sungchul Kim. Continuous-time dynamic network embeddings. In *WWW '18*, pp. 969–976, 2018. ISBN 9781450356404. doi: 10.1145/3184558.3191526. URL <https://doi.org/10.1145/3184558.3191526>.
- Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, and Charles E Leiserson. Evolvegc: Evolving graph convolutional networks for dynamic graphs. *arXiv:1902.10191*, 2019.

-
- Sarah Parisot, Sofia Ira Ktena, Enzo Ferrante, Matthew Lee, Ricardo Guerrero, Ben Glocker, and Daniel Rueckert. Disease prediction using graph convolutional networks: Application to autism spectrum disorder and alzheimer’s disease. *Med Image Anal*, 48:117–130, 2018.
- Yulong Pei, Jianpeng Zhang, GH Fletcher, and Mykola Pechenizkiy. Node classification in dynamic social networks. *AALTD*, pp. 54, 2016.
- Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. Deepwalk: Online learning of social representations. In *KDD ’14*, pp. 701–710, 2014. ISBN 9781450329569. doi: 10.1145/2623330.2623732. URL <https://doi.org/10.1145/2623330.2623732>.
- Siyuan Qi, Wenguan Wang, Baoxiong Jia, Jianbing Shen, and Song-Chun Zhu. Learning human-object interactions by graph parsing neural networks. In *ECCV*, pp. 401–417, 2018.
- Emanuele Rossi, Federico Monti, Michael M. Bronstein, and Pietro Liò. ncna classification with graph convolutional networks. In *KDD Workshop on Deep Learning on Graphs*, 2019.
- Emanuele Rossi, Fabrizio Frasca, Ben Chamberlain, Davide Eynard, Michael Bronstein, and Federico Monti. Sign: Scalable inception graph neural networks. *arXiv preprint arXiv:2004.11198*, 2020.
- Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, and Hao Yang. Dysat: Deep neural representation learning on dynamic graphs via self-attention networks. In *WSDM*, pp. 519–527, 2020.
- Youngjoo Seo, Michaël Defferrard, Pierre Vandergheynst, and Xavier Bresson. Structured sequence modeling with graph convolutional recurrent networks. *Lecture Notes in Computer Science*, pp. 362–373, 2018. ISSN 1611-3349. doi: 10.1007/978-3-030-04167-0_33. URL http://dx.doi.org/10.1007/978-3-030-04167-0_33.
- Umang Sharan and Jennifer Neville. Temporal-relational classifiers for prediction in evolving domains. In *ICDM*, pp. 540–549. IEEE, 2008.
- Uriel Singer, Ido Guy, and Kira Radinsky. Node embedding over temporal graphs. In *IJCAI*, pp. 4605–4612, 7 2019. doi: 10.24963/ijcai.2019/640. URL <https://doi.org/10.24963/ijcai.2019/640>.
- Rakshit Trivedi, Hanjun Dai, Yichen Wang, and Le Song. Know-evolve: Deep temporal reasoning for dynamic knowledge graphs. In *ICML*, pp. 3462–3471, 2017.
- Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, and Hongyuan Zha. Dyrep: Learning representations over dynamic graphs. In *ICLR*, 2019. URL <https://openreview.net/forum?id=HyePrhR5KX>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, pp. 5998–6008. 2017. URL <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>.
- Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *ICLR*, 2018.
- Kirill Veselkov et al. Hyperfoods: Machine intelligent mapping of cancer-beating molecules in foods. *Scientific Reports*, 9(1):1–12, 2019.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, R’emi Louf, Morgan Funtowicz, and Jamie Brew. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv:1910.03771*, 2019.
- Yu Xin, Zhi-Qiang Xie, and Jing Yang. An adaptive random walk sampling method on dynamic community detection. *Expert Systems with Applications*, 58:10–19, 2016.
- Chengjin Xu, Mojtaba Nayyeri, Fouad Alkhoury, Jens Lehmann, and Hamed Shariat Yazdi. Temporal knowledge graph completion based on time series gaussian embedding. *arXiv:1911.07893*, 2019.

-
- Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. Inductive representation learning on temporal graphs. In *ICLR*, 2020. URL <https://openreview.net/forum?id=rJeWlyHYwH>.
- Lin Yao, Luning Wang, Lv Pan, and Kai Yao. Link prediction based on common-neighbors for dynamic social network. *Procedia Computer Science*, 83:82–89, 2016.
- Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *KDD '18*, 2018.
- Bing Yu, Mengzhang Li, Jiyong Zhang, and Zhanxing Zhu. 3d graph convolutional networks with temporal graphs: A spatial information free framework for traffic forecasting. *arXiv:1903.00919*, 2019.
- Wenchao Yu, Wei Cheng, Charu C Aggarwal, Haifeng Chen, and Wei Wang. Link prediction with spatial and temporal consistency in dynamic networks. In *IJCAI*, pp. 3343–3349, 2017.
- Wenchao Yu, Wei Cheng, Charu C Aggarwal, Kai Zhang, Haifeng Chen, and Wei Wang. Netwalk: A flexible deep embedding approach for anomaly detection in dynamic networks. In *KDD '18*, pp. 2672–2681, 2018.
- Jiani Zhang, Xingjian Shi, Junyuan Xie, Hao Ma, Irwin King, and Dit-Yan Yeung. Gaan: Gated attention networks for learning on large and spatiotemporal graphs. *Conference on Uncertainty in Artificial Intelligence*, 2018.
- Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks. In *NIPS*, 2018.
- Lekui Zhou, Yang Yang, Xiang Ren, Fei Wu, and Yueting Zhuang. Dynamic network embedding by modeling triadic closure process. In *AAAI*, 2018.
- Jia Zhu, Qing Xie, and Eun Jung Chin. A hybrid time-series link prediction framework for large social network. In *DEXA*, pp. 345–359. Springer, 2012.
- Yu Zhu, Hao Li, Yikang Liao, Beidou Wang, Ziyu Guan, Haifeng Liu, and Deng Cai. What to do next: Modeling user behaviors by time-lstm. In *IJCAI*, volume 17, pp. 3602–3608, 2017.
- Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018.

A APPENDIX

A.1 DELETION EVENTS

The TGN frameworks also support edge and node deletions events.

In the case of an **edge deletion** event (i, j, t', t) where an edge between nodes i and j which was created at time t' is deleted at time t , two messages can be computed for the source and target nodes that respectively started and received the interaction:

$$\mathbf{m}_i(t) = \text{msg}_{s'}(\mathbf{s}_i(t^-), \mathbf{s}_j(t^-), \Delta t, \mathbf{e}_{ij}(t)), \quad \mathbf{m}_j(t) = \text{msg}_{d'}(\mathbf{s}_j(t^-), \mathbf{s}_i(t^-), \Delta t, \mathbf{e}_{ij}(t)) \quad (12)$$

In case of a **node deletion** event, we simply remove the node (and its incoming and outgoing edges) from the temporal graph so that when computing other nodes embedding this node is not used during the temporal graph attention. Additionally, it would be possible to compute a message from the node’s feature and memory and use it to update the memories of all its neighbors.

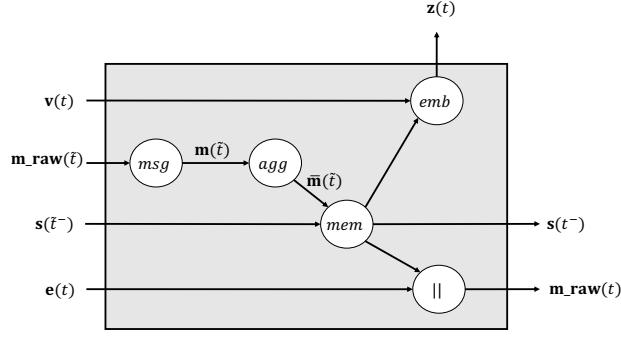


Figure 4: Schematic diagram of TGN. $\mathbf{m_raw}(t)$ is the raw message generated by event $\mathbf{e}(t)$, $\tilde{t}$ is the instant of time of the last event involving each node, and t^- the one immediately preceding t .

A.2 TGN TRAINING

When parallelizing the training of TGN, it is important to maintain the temporal dependencies between interactions. If the events on each node were independent, we could treat them as a sequence and train an RNN on each node independently using Back Propagation Through Time (BTPP). However, the graph structure introduces dependencies between the events (the update of a node depends on the current memory of other nodes) which prevent us from processing nodes in parallel. Previous methods either process the interactions one at a time, or use the t-batch (Kumar et al., 2019) training algorithm, which however does not satisfy temporal consistency when aggregating from the graph as in the case of TGN (since the update does not only depend on the memory of the other node involved the interaction, but also on the neighbors of the two nodes).

This issues motivate our training algorithm, which processes all interactions in batches following the chronological order. It stores the last message for each node in a message store, to process it before predicting the next interaction for the node. This allows the memory-related modules to receive a gradient. Algorithm 1 presents the pseudocode for TGN training, while Figure 4 shows a schematic diagram of TGN.

Algorithm 1: Training TGN

```

1  $\mathbf{s} \leftarrow \mathbf{0}$ ; // Initialize memory to zeros
2  $\mathbf{m\_raw} \leftarrow \{\}$ ; // Initialize raw messages
3 foreach batch  $(\mathbf{i}, \mathbf{j}, \mathbf{e}, \mathbf{t}) \in \text{training data}$  do
4    $\mathbf{n} \leftarrow \text{sample negatives}$ ;
5    $\mathbf{m} \leftarrow \text{msg}(\mathbf{m\_raw})$ ; // Compute messages from raw features1
6    $\bar{\mathbf{m}} \leftarrow \text{agg}(\mathbf{m})$ ; // Aggregate messages for the same nodes
7    $\hat{\mathbf{s}} \leftarrow \text{mem}(\bar{\mathbf{m}}, \mathbf{s})$ ; // Get updated memory
8    $\mathbf{z}_i, \mathbf{z}_j, \mathbf{z}_n \leftarrow \text{emb}_{\hat{\mathbf{s}}}(\mathbf{i}, \mathbf{t}), \text{emb}_{\hat{\mathbf{s}}}(\mathbf{j}, \mathbf{t}), \text{emb}_{\hat{\mathbf{s}}}(\mathbf{n}, \mathbf{t})$ ; // Compute node embeddings3
9    $\mathbf{p}_{\text{pos}}, \mathbf{p}_{\text{neg}} \leftarrow \text{dec}(\mathbf{z}_i, \mathbf{z}_j), \text{dec}(\mathbf{z}_i, \mathbf{z}_n)$ ; // Compute interactions probs
10   $l = \text{BCE}(\mathbf{p}_{\text{pos}}, \mathbf{p}_{\text{neg}})$ ; // Compute BCE loss
11   $\mathbf{m\_raw}_i, \mathbf{m\_raw}_j \leftarrow (\hat{\mathbf{s}}_i, \hat{\mathbf{s}}_j, \mathbf{t}, \mathbf{e}), (\hat{\mathbf{s}}_j, \hat{\mathbf{s}}_i, \mathbf{t}, \mathbf{e})$ ; // Compute raw messages
12   $\mathbf{m\_raw} \leftarrow \text{store\_raw\_messages}(\mathbf{m\_raw}, \mathbf{m\_raw}_i, \mathbf{m\_raw}_j)$ ; // Store raw
    messages
13   $\mathbf{s}_i, \mathbf{s}_j \leftarrow \hat{\mathbf{s}}_i, \hat{\mathbf{s}}_j$ ; // Store updated memory for sources and
    destinations
14 end
```

¹For the sake of clarity, we use the same message function for both sources and destination.

²We denote with $\text{emb}_{\hat{\mathbf{s}}}$ an embedding layer that operates on the updated version of the memory $\hat{\mathbf{s}}$.

A.3 DATASETS

Reddit and Wikipedia are bipartite interaction graphs. In the Reddit dataset, users and sub-reddits are nodes, and an interaction occurs when a user writes a post to the sub-reddit. In the Wikipedia dataset, users and pages are nodes, and an interaction represents a user editing a page. In both aforementioned datasets, the interactions are represented by text features (of a post or page edit, respectively), and labels represent whether a user is banned. Both interactions and labels are time-stamped.

The Twitter dataset is a non-bipartite graph released as part of the 2020 RecSys Challenge (Belli et al., 2020). Nodes are users and interactions are retweets. The features of an interaction are a BERT-based (Wolf et al., 2019) vector representation of the text of the retweet.

Node features are not present in any of these datasets, and we therefore assign the same zero feature vector to all nodes. Moreover, While our framework is general and in section 3.1 we showed how it can process any type of event, these three datasets only contain the edge creation (interaction) event type. Creating and evaluation of datasets with a wider variety of events is left as future work.

The statistics of the three datasets are reported in table 4.

Table 4: Statistics of the datasets used in the experiments.

	Wikipedia	Reddit	Twitter
# Nodes	9,227	11,000	8,861
# Edges	157,474	672,447	119,872
# Edge features	172	172	768
# Edge features type	LIWC	LIWC	BERT
Timespan	30 days	30 days	7 days
Chronological Split	70%-15%-15%	70%-15%-15%	70%-15%-15%
# Nodes with dynamic labels	217	366	–

Twitter Dataset Generation To generate the Twitter dataset we started with the snapshot of the Recsys Challenge training data on 2020/09/06. We filtered the data to include only retweet edges (discarding other types of interactions) where the timestamp was present. This left approximately 10% of the edges in the original dataset. We then filtered the retweet multi-graph (users can be connected by multiple retweets) to only include the largest connected component. Finally, we filtered the graph to only the top 5,000 nodes in-degree and the top 5,000 by out-degree, ending up with 8,861 nodes since some nodes were in both sets.

A.4 ADDITIONAL EXPERIMENTAL SETTINGS AND RESULTS

Hyperparameters For the all datasets, we use the Adam optimizer with a learning rate of 0.0001, a batch size of 200 for both training, validation and testing, and early stopping with a patience of 5. We sample an equal amount of negatives to the positive interactions, and use *average precision* as reference metric. Additional hyperparameters used for both future edge prediction and dynamic node classification are reported in table 5. For all the graph embedding modules we use neighbors sampling (Hamilton et al., 2017b) (i.e. only aggregate from k neighbors) since it improves the efficiency of the model without losing in accuracy. In particular, the sampled edges are the k most recent ones, rather than the traditional approach of sampling them uniformly, since we found it to perform much better (see Figure 5). All experiments and timings are conducted on an AWS p3.16xlarge machine and the results are averaged over 10 runs. The code will be made available for all our experiments to be reproduced.

Baselines Results Our results for GAE (Kipf & Welling, 2016), VGAE (Kipf & Welling, 2016), DeepWalk (Perozzi et al., 2014), Node2Vec (Grover & Leskovec, 2016), GAT (Velickovic et al., 2018) and GraphSAGE (Hamilton et al., 2017b), CTDNE (Nguyen et al., 2018) and TGAT (Xu et al., 2020) are taken directly from the TGAT paper (Xu et al., 2020). For Jodie (Kumar et al., 2019) and DyRep (Trivedi et al., 2019), in order to make the comparison as fair as possible, we implement our own version in PyTorch as a specific case of our tgn framework. For Jodie we simply use the

Table 5: Model Hyperparameters.

	Value
Memory Dimension	172
Node Embedding Dimension	100
Time Embedding Dimension	100
# Attention Heads	2
Dropout	0.1

time embedding module, while for DyRep we augment the messages with the result of a temporal graph attention performed on the destination’s neighborhood. For both we use a vanilla RNN as the memory updater module.

A.4.1 NEIGHBOR SAMPLING: UNIFORM VS MOST RECENT

When performing neighborhood sampling (Hamilton et al., 2017a) in static graphs, nodes are usually sampled uniformly. While this strategy is also possible for dynamic graphs, it turns out that the most recent edges are often the most informative. In Figure 5 we compare two TGN-attn models (see Table 1) with either uniform or most recent neighbor sampling, which shows that a model which samples the most recent edges obtains higher performances.

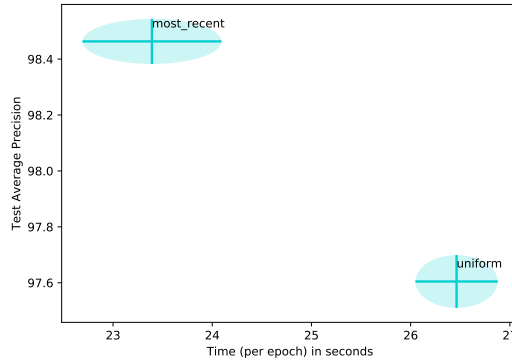


Figure 5: Comparison of two TGN-attn models using different neighbor sampling strategies (when sampling 10 neighbors). Sampling the most recent edges clearly outperforms uniform sampling. Means and standard deviations (visualized as ellipses) were computed over 10 runs.